Marc Gamell

Informe de la pràctica 1 Blink i Activitats Complementàries

Introducció

Aquesta primera pràctica de l'assignatura Processadors Digitals serveix com a presa de contacte amb l'ecosistema de desenvolupament basat en el microcontrolador ESP32-S3. Al llarg de l'exercici, s'exploren des de les operacions més bàsiques de control de sortides digitals fins a la gestió de llibreries i la lectura de sensors interns del propi xip.

1.1. Objectius

La pràctica s'ha dividit en dues fases amb els següents objectius específics:

Part Principal (Blink i Sensors):

- Configurar l'entorn de treball PlatformIO per a la placa ESP32-S3.
- Programar el parpelleig periòdic d'un LED mitjançant sortides digitals.
- Utilitzar el terminal sèrie per a la depuració de programes.
- Implementar la lectura d'un senyal analògic (ADC) i la seva conversió a una sortida de potència simulada (PWM).
- Monitoritzar la temperatura interna del silici del processador.

Part Complementària (RGB i Maquinari):

- Aprendre la gestió de llibreries externes en l'entorn PlatformIO.
- Controlar un LED intel·ligent de tipus RGB (NeoPixel/WS2812) integrat en la placa.
- Interrogar el microcontrolador per obtenir informació tècnica del maquinari (model, revisió, memòria flash i adreça MAC).

1.2. Material utilitzat

Per a la realització d'aquesta pràctica s'ha utilitzat el següent material:

Maquinari (Hardware):

- **Microcontrolador:** Placa de desenvolupament model **YD-ESP32-S3**, compatible amb el model ESP32-S3-DevKitC-1 d'Espressif.
- **LEDs:** LED estàndard connectat al pin 35 i LED RGB (NeoPixel) connectat al pin 48.
- **Cablejat:** Cable de dades USB-C per a la programació i alimentació de la placa.
- **Perifèrics interns:** Convertidor Analògic-Digital (ADC) i sensor de temperatura integrat.

Programari (Software):

- **Editor de codi:** Visual Studio Code amb l'extensió **PlatformIO**.
- **Llenguatge de programació:** C++ basat en el framework d'**Arduino**.
- **Llibreries externes:** Adafruit_NeoPixel per al control del LED multicolor.

1.3. Descripció del Maquinari

L'**ESP32-S3** és un sistema en un xip (SoC) d'alt rendiment que integra connectivitat Wi-Fi de 2,4 GHz i Bluetooth Low Energy (BLE). Està basat en un microprocessador Xtensa® Dual-Core de 32 bits.

A diferència de models anteriors, l'S3 destaca per la seva gran quantitat de pins de sortida configurables i per la inclusió d'instruccions d'acceleració per a intel·ligència artificial. En aquesta pràctica, aprofitarem la seva versatilitat per gestionar tant entrades analògiques de 12 bits com protocols de comunicació sèrie per al LED RGB.

Desenvolupament i Implementació

Per assolir l'objectiu principal de la pràctica, que és produir el parpelleig periòdic d'un LED, la programació s'ha dividit en la configuració de l'entorn de desenvolupament i la implementació del codi principal. D'acord amb els requisits, també s'utilitzarà la sortida sèrie per depurar i fer un seguiment de l'estat del programa.

1. Configuració de l'Entorn (platformio.ini)

L'arxiu platformio.ini és el fitxer de configuració del nostre projecte. Aquest arxiu és fonamental perquè s'encarrega de dir-li a l'entorn de desenvolupament quina placa física estem utilitzant i sota quins paràmetres s'ha de compilar el codi. Les línies de configuració utilitzades són les següents:

- **[env:esp32-s3-devkitm-1]**: Aquesta línia defineix el nom de l'entorn de treball. Indica que estem preparant una configuració específica per a aquesta placa.
- **platform = espressif32**: Defineix l'arquitectura del microcontrolador. Indica que estem treballant amb la família de xips de l'empresa Espressif (els creadors de l'ESP32).
- **board = esp32-s3-devkitm-1**: Especifica el model exacte de la placa de desenvolupament que estem fent servir. D'aquesta manera, el compilador sap quina distribució de pins i característiques de maquinari té.
- **framework = arduino**: Indica que utilitzarem el marc de treball (framework) d'Arduino. Això ens permet programar el nostre ESP32 utilitzant les funcions i llibreries d'Arduino, la qual cosa simplifica molt la lectura i escriptura del codi.
- **monitor_speed = 115200**: Configura la velocitat del monitor sèrie a 115200 bauds (bits per segon). És un paràmetre crític, ja que aquesta velocitat ha de coincidir exactament amb la que es defineix en el codi per poder llegir correctament els missatges a l'ordinador.

2. Codi Principal: Control del LED (main_practica1.cpp)

Pel que fa a l'arxiu principal, en aquesta primera etapa ens hem centrat exclusivament en les instruccions que controlen l'encesa i apagada del LED i els missatges enviats pel terminal.

Definicions inicials:

- **#include <Arduino.h>**: Importa la llibreria base per poder utilitzar el llenguatge i les funcions integrades d'Arduino.
- **#define LED_PIN 35**: Associa de forma constant el nom LED_PIN al número 35. Això és una bona pràctica de programació; si en un futur calgués connectar el LED a un altre pin, només hauríem de canviar aquest número un sol cop en lloc de buscar-lo per tot el codi.

Funció setup():

Aquesta funció conté el codi de preparació i s'executa una única vegada just quan s'engega o es reinicia el microcontrolador.

- **Serial.begin(115200);**: Aquesta línia s'encarrega d'iniciar el terminal sèrie. Estableix la velocitat de comunicació a 115200 bauds per parlar amb l'ordinador, la qual coincideix exactament amb el paràmetre monitor_speed de l'arxiu de configuració.
- **pinMode(LED_PIN, OUTPUT);**: Serveix per iniciar el pin del LED com a sortida. Informa al microcontrolador que el pin 35 no rebrà dades, sinó que se n'encarregarà de subministrar voltatge.

Funció loop(): Aquesta funció és el nucli de l'aplicació i es reprodueix de forma contínua, actuant com un bucle infinit.

- **digitalWrite(LED_PIN, HIGH);**: Envia un nivell lògic alt (voltatge) al pin, la qual cosa permet encendre el LED.
- **Serial.println("--- INICI CICLE ---");**: Treu per la sortida del port sèrie un missatge per avisar l'usuari. Aquesta sortida confirma visualment per pantalla que el cicle de parpelleig comença i el LED està encès.
- **delay(500);**: Genera una espera de 500 mil·lisegons (mig segon). Durant aquest temps, el processador no canvia d'estat i manté el LED encès.
- **digitalWrite(LED_PIN, LOW);**: Envia un nivell lògic baix (0 volts) al pin amb l'objectiu d'apagar el LED.
- **Serial.println("LED OFF");**: Treu pel port sèrie el missatge corresponent informant que el LED s'acaba d'apagar.
- **delay(500);**: Produeix una nova espera de 500 mil·lisegons amb el LED apagat abans que la funció loop() torni a començar des de la primera línia.

3. Activitat Complementària: Conversió A/D i Sortida Analògica (PWM)

Com a part dels exercicis voluntaris de millora , s'ha d'implementar la lectura d'un valor d'un convertidor A/D (Analògic a Digital) i treure'l per un altre pin com a sortida D/A (Digital a Analògic).

Cal destacar un aspecte tècnic molt important: el xip ESP32-S3 no disposa de convertidors D/A de maquinari pur com altres models anteriors. Per resoldre això, s'utilitza una tècnica anomenada **PWM** (Modulació per Amplada de Polsos), que permet simular un senyal analògic variant ràpidament un senyal digital.

A continuació es detalla com s'ha implementat això en el codi:

Definicions inicials:

- **#define ADC_PIN 1**: S'assigna el pin número 1 per actuar com a entrada analògica. Aquest pin llegirà variacions de voltatge, provinents, per exemple, d'un potenciòmetre o un sensor.
- **#define PWM_PIN 36**: S'assigna el pin 36 per ser la sortida analògica simulada mitjançant PWM. Tal com indica el codi, en el model ESP32-S3, qualsevol pin de propòsit general pot realitzar funcions de PWM.

Funció setup() (Configuració):

- **analogReadResolution(12);**: Aquesta línia configura la precisió (resolució) amb la qual el microcontrolador llegeix l'entrada analògica a 12 bits. Un senyal analògic és continu, i el microcontrolador l'ha de traduir a nombres digitals per entendre'l. Amb 12 bits de resolució, el xip transforma el voltatge d'entrada en una escala de valors que va des del 0 fins al 4095.

Funció loop() (Execució contínua):

- **int adcValue = analogRead(ADC_PIN);**: Aquesta línia pren la lectura de voltatge del pin 1 en aquell mateix instant i guarda el resultat numèric (entre 0 i 4095) en una variable anomenada adcValue.
- **int pwmValue = map(adcValue, 0, 4095, 0, 255);**: Aquest pas és crucial. La resolució per defecte per generar un senyal PWM en el framework d'Arduino és de 8 bits (valors de 0 a 255). Com que el nostre valor llegit està en una escala diferent (0 a 4095), utilitzem la funció map() per fer una conversió matemàtica proporcional. Per exemple, si llegim un valor de 2047 (la meitat de l'entrada), la funció ho convertirà automàticament a 127 (la meitat de la sortida PWM). El resultat es guarda a pwmValue.
- **analogWrite(PWM_PIN, pwmValue);**: Aquesta instrucció envia finalment el senyal de sortida pel pin 36. L'amplada del pols PWM serà proporcional al valor que hem calculat al pas anterior, comportant-se efectivament com una sortida D/A.
- **Serial.print(...) i Serial.println(...)**: Aquestes línies de codi s'encarreguen d'enviar les dades cap al monitor sèrie de l'ordinador. Concretament, mostren el text "Entrada ADC: " seguit del valor cru llegit a 12 bits, i després " | Sortida PWM: " seguit del valor escalat a 8 bits. Això resulta fonamental per depurar el programa i verificar visualment que la conversió matemàtica entre les dues escales es realitza correctament en tot moment.

4. Lectura del Sensor de Temperatura Intern

L'última funcionalitat implementada en el programa és la lectura del sensor de temperatura intern del xip. Aquesta és una característica avançada de l'ESP32-S3 que permet monitoritzar l'estat del propi silici, sent un dels exercicis proposats per a la millora de la nota.

Les línies de codi encarregades d'aquesta tasca són les següents:

- `float temp = temperatureRead();`: Aquesta instrucció crida una funció específica del framework d'Espressif que llegeix el sensor tèrmic integrat dins del microcontrolador. El valor obtingut es guarda en una variable de tipus `float` (coma flotant) anomenada `temp`, ja que la temperatura es mesura amb decimals per a una major precisió. Cal destacar que aquesta mesura correspon a la temperatura de funcionament del xip i no a la temperatura ambient.
- `Serial.print("Temp. Interna ESP32-S3: ");`: Envia al terminal el text indicatiu per a que l'usuari sàpiga quina dada s'està mostrant a continuació.
- `Serial.print(temp);`: Imprimeix el valor numèric emmagatzemat a la variable `temp`.
- `Serial.println(" °C");`: Afegeix la unitat de mesura (graus Celsius) i realitza un salt de línia al terminal de sortida per a que la següent informació aparegui clarament a sota.

Sortida pel Terminal de Control

Tota la informació recollida tant pel control del LED com per les activitats de millora (ADC i temperatura) es visualitza de forma estructurada al terminal sèrie de l'ordinador. Cada cicle del programa (que dura aproximadament un segon a causa dels dos retards de 500 ms) mostra una sortida similar a aquesta:

1. `--- INICI CICLE ---`: Indica que el LED s'ha encès i comença el bucle.
2. `Entrada ADC: [Valor] | Sortida PWM: [Valor]`: Mostra la lectura en temps real del pin analògic i la seva traducció proporcional a la sortida.
3. `Temp. Interna ESP32-S3: [Valor] °C`: Mostra l'estat tèrmic actual del processador.
4. `LED OFF`: Indica que el cicle de parpelleig ha completat la fase d'encesa i el LED s'ha apagat.

Aquesta sortida és vital per a la depuració, ja que permet confirmar que el processador està executant totes les tasques correctament sense necessitat d'utilitzar equips de mesura externs.

Anàlisi del Temps Lliure del Processador

Per respondre a la pregunta sobre quin és el temps lliure que té el processador en el programa que s'ha realitzat, és necessari analitzar el cicle d'execució del nostre bucle principal (`loop()`).

En el món dels microcontroladors, les instruccions s'executen a velocitats extremadament altes (l'ESP32-S3 pot operar a velocitats de fins a 240 MHz, és a dir, milions d'operacions per segon). Això vol dir que llegir el sensor analògic, fer la conversió matemàtica, llegir la temperatura interna i enviar les dades pel port sèrie, triga només una fracció minúscula de temps (uns pocs mil·lisegons o fins i tot microsegons).

No obstant això, dins del nostre codi tenim dues instruccions de pausa molt importants:

- `delay(500);` (amb el LED encès)
- `delay(500);` (amb el LED apagat)

Càlcul de l'ocupació: Cada cicle complet del programa dura aproximadament 1000 mil·lisegons (1 segon) de pausa, més el petit temps que triguen a executar-se la resta de línies de codi.

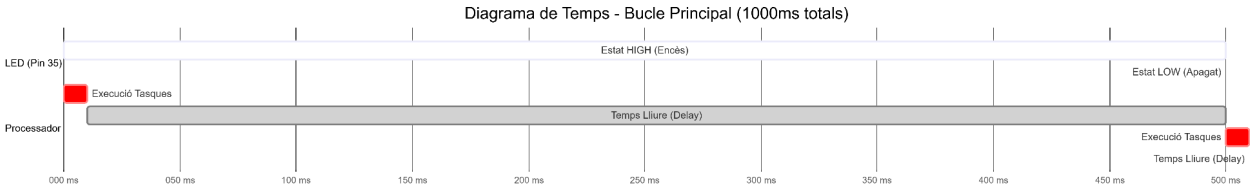
- **Temps actiu (treballant):** < 1% del temps total del cicle.
- **Temps lliure (esperant):** > 99% del temps total del cicle.

Què significa aquest "temps lliure"? Tot i que diem que el processador té més d'un 99% de "temps lliure", la instrucció `delay()` utilitzada en el framework d'Arduino és el que s'anomena una **funció bloquejant**. Durant aquests 500 mil·lisegons d'espera, el processador no fa pràcticament res més que "comptar el temps" fins que s'acabi el termini.

En resum, el processador està ociós gairebé tot el temps, ja que la càrrega computacional de la pràctica és extremadament baixa. En aplicacions més avançades on no ens podem permetre tenir el processador aturat esperant (per exemple, si haguéssim d'atendre ràpidament un botó d'emergència), s'haurien d'utilitzar mètodes no bloquejants (com la funció `millis()`) o interrupcions de maquinari, per tal d'aprofitar tot aquest temps lliure executant altres tasques simultànies.

Diagrames de temps i flux

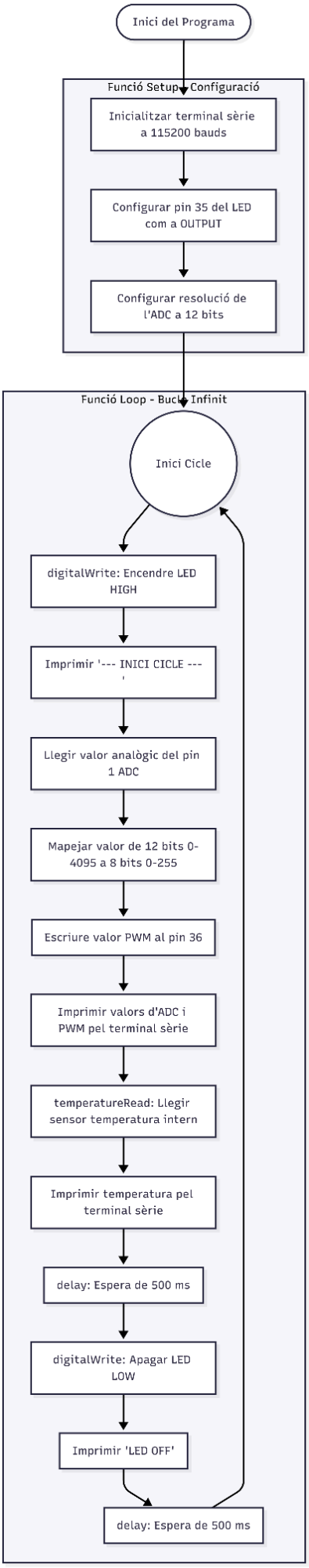
Diagrama de Temps



En aquest diagrama podem observar com es distribueix el temps durant un cicle sencer d'un segon (1000 mil·lisegons). A la secció superior es mostra l'estat físic del LED, que roman 500 ms encès i 500 ms apagat.

A la secció inferior es fa evident la resposta sobre el temps lliure del processador. Es pot veure (marcat en vermell/crític) que les tasques de lectura dels sensors i enviament de dades pel port sèrie ocupen una franja de temps minúscula (representada de forma aproximada com 10 ms, tot i que a la pràctica són microsegons). La resta del temps (les caixes blaves), el processador es troba executant la funció `delay()`, romanent ociós i a l'espera fins que acabi la temporització.

Diagrama de flux



Desenvolupament de l'Activitat Complementària

Aquesta pràctica complementària té com a objectius principals el control del LED RGB integrat a la placa ESP32-S3 (del tipus NeoPixel) i la lectura de les característiques físiques del maquinari.

1. Explicació del Codi (main_practica1complementaria.cpp)

Per assolir aquests objectius, el programa s'estructura de la següent manera:

Inclusió de llibreries i definicions globals:

- **#include <Adafruit_NeoPixel.h>**: Aquesta línia és fonamental. El LED RGB d'aquesta placa funciona mitjançant un bus sèrie específic. Per no haver de programar aquest protocol de comunicació des de zero, importem aquesta llibreria externa prèviament instal·lada a PlatformIO.
- **#define PIN 48** i **#define NUMPIXELS 1**: S'indica que el LED intel·ligent està connectat al pin 48 i que només controlarem un únic LED.
- **Adafruit_NeoPixel pixels(...)**: Es crea l'objecte (la instància) que ens permetrà interactuar amb el LED utilitzant les funcions de la llibreria.

Funció setup() (Configuració i Maquinari):

- **pixels.begin();**: Inicialitza la comunicació amb el LED NeoPixel.
- **pixels.setBrightness(10);**: Aquesta instrucció limita la potència màxima del LED a un valor molt baix (10 sobre 255). Amb això es dona compliment estricte al requisit de l'enunciat de mostrar els colors amb una **intensitat lumínica mínima**.
- **Informació del xip:** Tota la part central del setup es dedica a cridar funcions internes (com ESP.getChipModel(), ESP.getChipCores(), o esp_chip_info()). Aquest bloc s'encarrega d'interrogar el microcontrolador sobre les seves pròpies capacitats (model, nuclis, memòria flash, connectivitat WiFi/Bluetooth i adreça MAC) i mostrar-ho pel terminal.

Funció loop() (Cicle de colors): El bucle principal està dissenyat per complir el requisit de visualitzar els 6 colors bàsics (RGBCMY) seguits del color blanc durant 1 segon cadascun.

- **pixels.setPixelColor(0, pixels.Color(R, G, B));**: Defineix el color barrejant les tres components (Red, Green, Blue) amb valors de 0 a 255. Per exemple, per fer l'**Amarillo** (groc), es posa al màxim el vermell i el verd (255, 255, 0).
- **pixels.show();**: Envia les dades configurades anteriorment al component físic perquè s'il·lumini amb el color escollit.
- **delay(DELAYVAL);**: Com que DELAYVAL està definit a 1000, el microcontrolador manté el color actual durant exactament 1 segon abans de passar al següent.

2. Anàlisi del Temps Lliure del Processador (Pràctica Complementària)

Al igual que en la pràctica principal del blink, l'ús de la CPU en aquest codi ens revela un processador altament desocupat.

Podem dividir l'execució d'un cicle sencer del loop() en 7 etapes (una per cada color: Vermell, Verd, Blau, Cian, Magenta, Groc i Blanc). Dins de cada etapa, el processador realitza el següent treball:

1. Calcula i empaqueta els valors RGB del color.
2. Envia la trama de dades al LED (funció pixels.show()).
3. Envia el nom del color pel port sèrie cap a l'ordinador.

Tot aquest "esforç computacional" li pren a l'ESP32-S3 només una fracció minúscula de mil·lisegon a causa de la seva alta velocitat de processament.

Immediatament després, el codi es troba amb la instrucció **delay(1000)**. Durant aquest segon sencer, el processador entra en estat de bloqueig i no fa absolutament cap tasca. Com que això es repeteix per cadascun dels 7 colors, per cada cicle sencer de 7 segons, el processador treballa durant uns pocs mil·lisegons en total.

Conclusió:

El temps lliure del processador en aquesta pràctica complementària és pràcticament del **99,9%**. Tanmateix, a l'utilitzar instruccions de retard bloquejants (delay()), aquest temps "lliure" és temps mort on el microcontrolador està paralitzat i no pot atendre cap altra tasca ni llegir cap altre sensor mentre espera que passi el segon de rigor.

Conclusions

Aquesta primera pràctica ha complert amb èxit tots els objectius plantejats, servint com una excel·lent introducció teoricopràctica a l'arquitectura del microcontrolador ESP32-S3 i al seu entorn de desenvolupament. A través de la implementació dels diferents exercicis, s'han extret les següents conclusions fonamentals:

- **Gestió eficient de l'entorn de treball:** S'ha comprovat la diferència metodològica entre l'IDE clàssic d'Arduino i PlatformIO. S'ha après que gestionar les llibreries de forma individualitzada per a cada projecte evita els conflictes de versions que sovint ocorren quan s'actualitzen llibreries de manera global. Això resulta en un entorn de programació més robust i reproduïble.
- **Interacció amb el maquinari físic i les seves limitacions:** Hem passat del control bàsic de senyals digitals (encendre i apagar un LED) a l'adquisició de dades analògiques de 12 bits. Un aprenentatge clau ha estat entendre com adaptar-se al maquinari disponible: atès que l'ESP32-S3 no disposa d'un convertidor D/A de maquinari pur, hem hagut d'aplicar solucions de programari, utilitzant la funció map() per escalar els valors i la modulació PWM per simular el senyal de sortida.
- **Aprofitament de recursos externs i diagnòstic intern:** L'ús del terminal sèrie ha demostrat ser una eina imprescindible per a la depuració, permetent visualitzar tant

les dades dels sensors com la informació vital del xip (revisió, adreça MAC, memòria). D'altra banda, la integració del LED RGB WS2812 ha evidenciat la facilitat d'ampliar les capacitats del microcontrolador mitjançant l'ús de llibreries de tercers (Adafruit_NeoPixel).

- **Reflexió sobre el temps de processament:** La conclusió més crítica a nivell d'arquitectura de computadors és l'anàlisi de l'ocupació de la CPU. Hem constatat que, malgrat la gran potència de l'ESP32-S3, en programes seqüencials senzills el processador roman inactiu pràcticament el 99,9% del temps. L'ús intensiu de funcions bloquejants com delay() paralitza l'execució i suposa un desaprofitament dels recursos. Això ens indica que, per a projectes futurs i sistemes en temps real, serà necessari explorar tècniques de programació no bloquejants (com millis()) o basades en interrupcions per poder realitzar tasques de forma concurrent i eficient.

En definitiva, aquesta pràctica estableix una base sòlida per entendre la relació directa entre les instruccions de codi i el comportament electrònic físic, preparant el terreny per afrontar sistemes digitals més complexos al llarg de l'assignatura.